Jala University

OdeolA

Oscar Steven Muñoz Fajardo

Audio compression, transcription, and summarization using concurrency, synchronization, and asynchronous programming.

Agenda



1. Problem, Scope and Objective
2. System Architecture
3. Execution Pipeline
4. Concurrency and Synchronization
5. Asynchronous Programming
6. Data Flow and Persistence
7. Demo

Problem, Scope and Objective

Process audio files without keeping the HTTP request waiting for heavy operations. (FFmpeg, Whisper, Llama — qwen2.5 model)

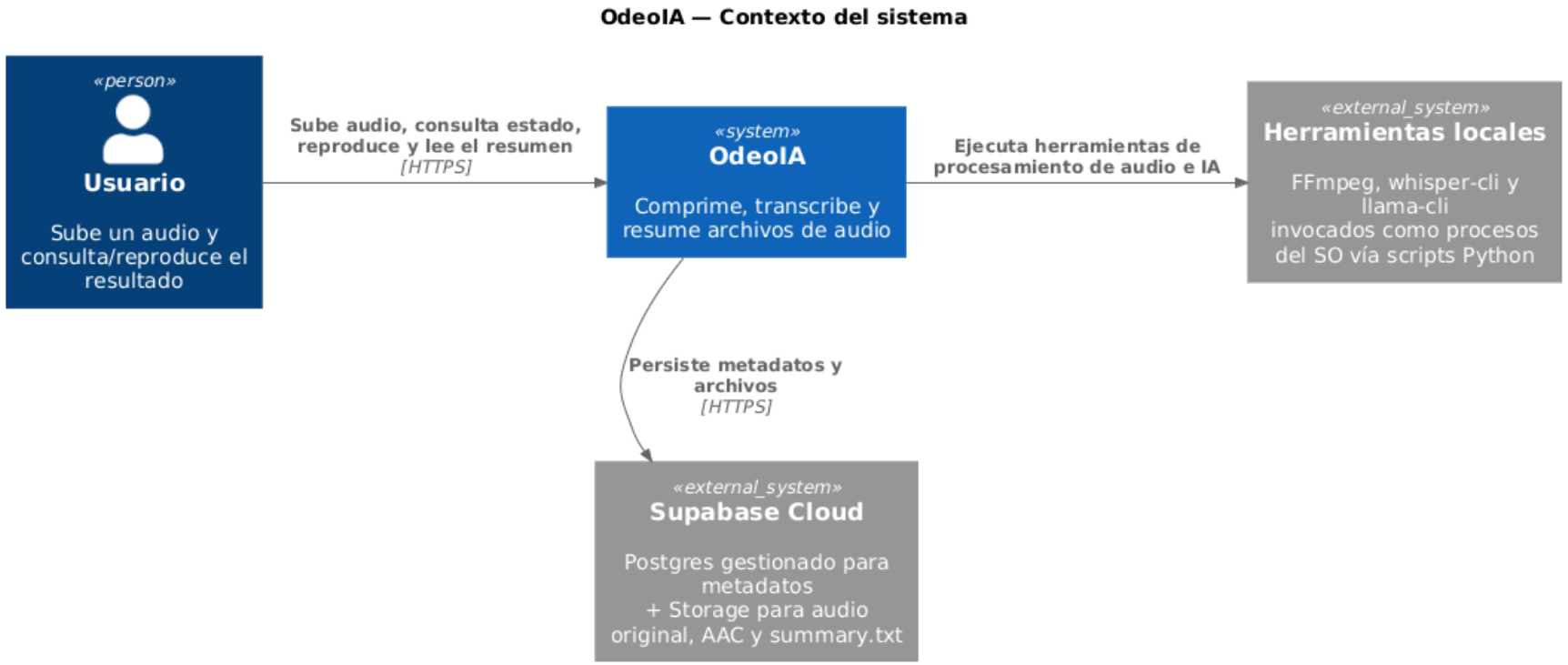
Scope

MP3 / WAV → Compression + Transcription → Summary

Objective

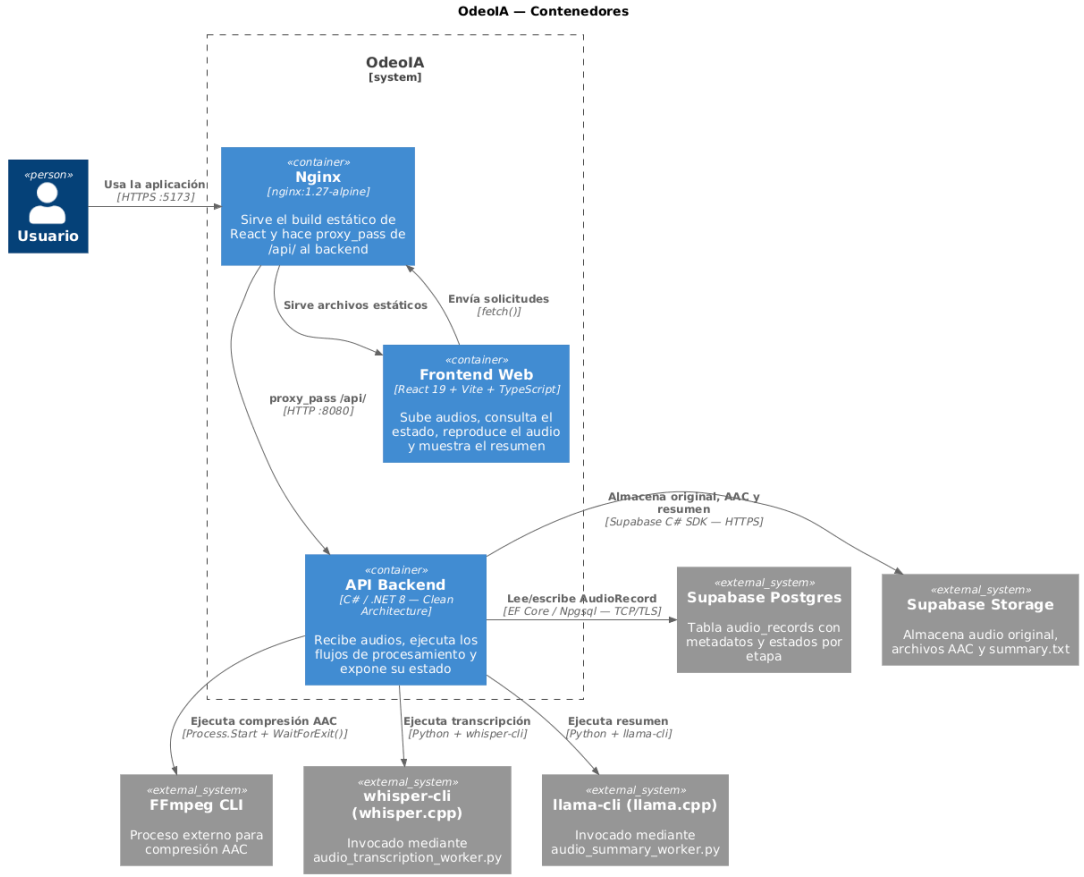
Apply concurrency, synchronization, and asynchronous programming where they are actually needed.

Architecture — System Context



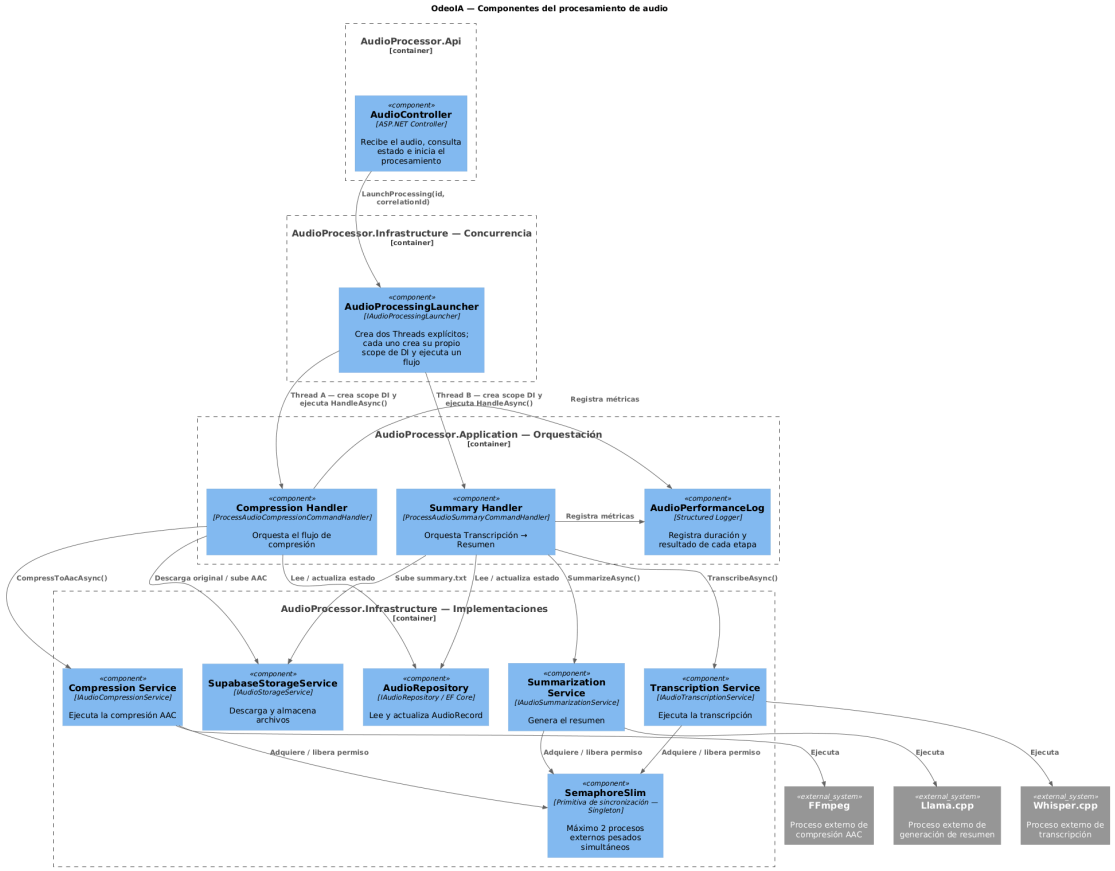
View diagram

Architecture — Containers



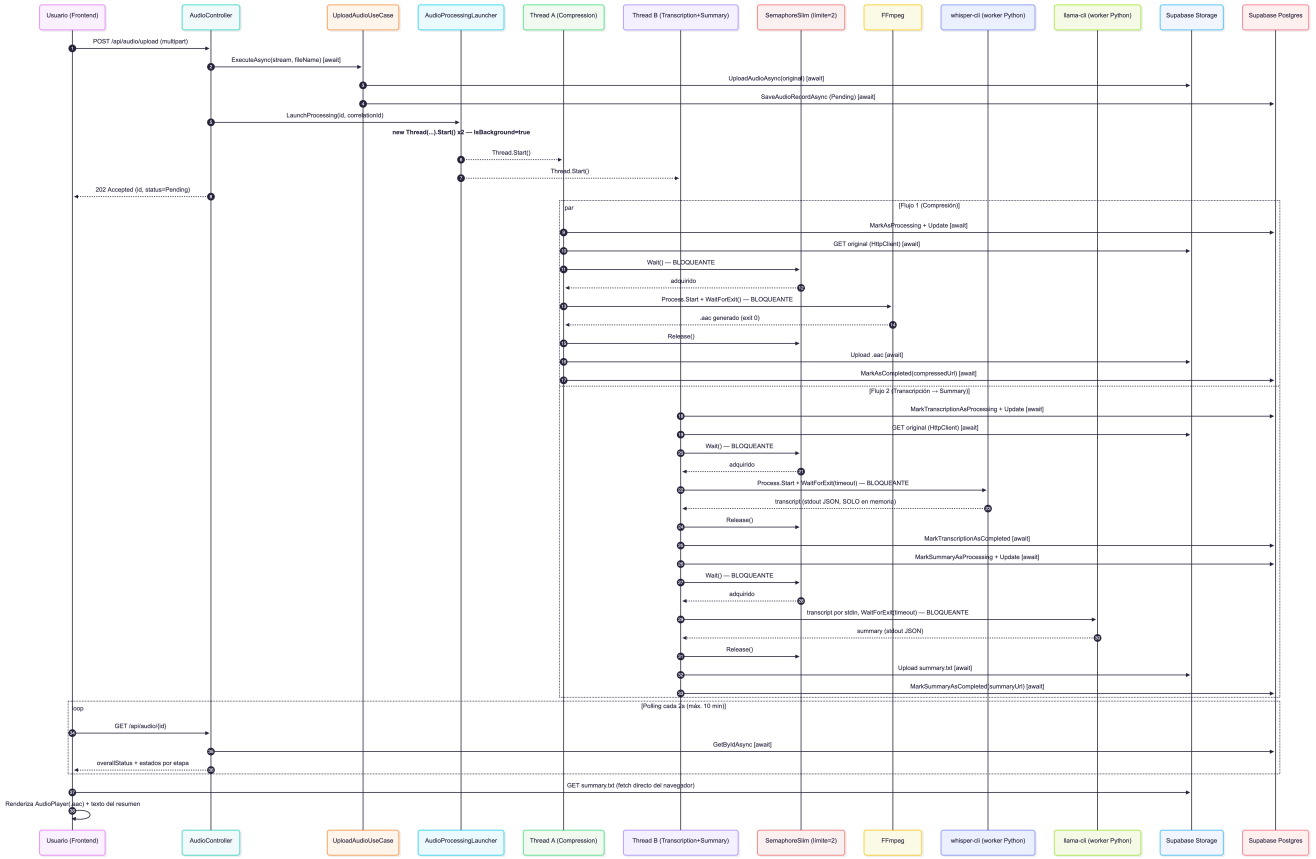
View diagram

Architecture — Components



View diagram

Execution Pipeline



View diagram

Concurrency — Thread

Two independent processing flows.

Thread A → Compression

Thread B → Transcription → Summary

Both flows can progress concurrently.

Synchronization — SemaphoreSlim

FFmpeg · Whisper · Llama

Maximum of 2 heavy external processes running simultaneously.

Thread enables concurrency.

SemaphoreSlim controls how much concurrency I allow.

Asynchrony — async / await



Asynchronous I/O

Storage

Database

HTTP

Streams

Blocking

FFmpeg

Whisper

Llama

WaitForExit()

Data Flow and Persistence

Original Audio → Storage

Compressed AAC → Storage

Transcription → In Memory

Summary → Storage

Jala University

Demo

OdeoIA in action